El papel del sistema operativo en un servidor de juego multijugador

Buenas tardes. Hoy les voy a presentar el papel del sistema operativo en un servidor de un videojuego multijugador, concretamente videojuegos de acción en tiempo real, como pueden ser shooters o juegos de pelea.

En este tipo de videojuegos se necesita un servidor donde posiblemente haya miles de personas jugando al mismo tiempo, además estas personas interactúan entre sí y la sincronización entre lo que los jugadores ven y lo que pasa realmente en el servidor, es de vital importancia para la buena experiencia del jugador. Así que la relevancia del buen uso del sistema operativo es importante, para que este no sea el cuello de botella del programa.

En la mayoría de los juegos multijugador el servidor es autoritativo, es decir, los clientes solo mandan sus entradas y es el servidor quien decide qué pasó realmente. Esto hace que los clientes se comuniquen únicamente con el sistema pero no entre ellos, lo que evita que el jugador pueda hacer cierto tipo de trampas, pero concentra gran parte del trabajo en el servidor.

Este trabajo se puede dividir en dos, el primero es la gestión de entradas y salidas, que son las miles de conexiones que mandan los usuarios, el segundo es de CPU, donde tenemos administrar esas conexiones y simular el mundo a intervalos fijos.

Gestión de conexiones

La primera forma de atender conexiones es la más intuitiva, un hilo o un proceso por jugador. De esta forma el código es secuencial y fácil de escribir, cada hilo se bloquea en recv() (función que se utiliza para recibir datos de un socket), donde el kernel lo saca de la cola de listos hasta que llegan datos al socket. El problema es el coste de cada hilo al sistema operativo.

Primero, crear un hilo implica reservar memoria para su pila y contexto de ejecución, una pila de hilo en Linux reserva por defecto 8 MB de espacio de direcciones (el límite de pila del proceso). Es memoria virtual y la física se asigna al usarla, pero el kernel además mantiene estructuras por hilo. Por ejemplo, si cada cliente consume 1 MB, mil clientes ya suman unos 1000 MB. Además con un hilo por conexión, buena parte de ese consumo es memoria que casi siempre está inactiva, porque la mayoría de los jugadores envía muy pocos datos entre un tick y otro.

Segundo, crear y destruir hilos cuesta tiempo, y con la cantidad que manejamos el rendimiento cae por cambios de contexto excesivos, cada cambio guarda y restaura registros, cambia de pila y deja la caché y el TLB poco útiles para el hilo que entra, lo que provoca que el sistema pase más tiempo alternando entre hilos que avanzando en el proceso.

Tercero, todos esos hilos tocan el mismo escenario del juego, así que aparecen candados y contención, lo que aumenta mucho la complejidad.

La alternativa es la multiplexación de E/S, es decir, en lugar de un hilo esperando por cada socket, un hilo le pregunta al kernel cuáles sockets están listos y atiende únicamente esos. En Linux se usa epoll, primero se crea una instancia con epoll_create, se registra cada descriptor una sola vez con epoll_ctl, y epoll_wait devuelve los que están listos. Tiene dos modos. En el modo por nivel, avisa de datos pendientes, mientras que en el modo por flanco avisa solo al cambiar de estado, exige descriptores no bloqueantes y leer hasta que `read` devuelva `EAGAIN`; si no, pueden quedar datos sin atender.

Hasta aquí, todos los mecanismos responden a una pregunta: ¿este socket está listo? Esto es el modelo de disponibilidad: el sistema avisa que se puede leer y luego el programa hace la lectura. Pero también es necesario utilizar el modelo de finalización. El programa pide la operación y el sistema avisa cuando ya terminó.

En Linux, la versión de esta idea es `io_uring`. Usa dos colas en memoria compartida entre el programa y el kernel: una de envío y una de finalización. El programa escribe varias peticiones y las entrega con una sola llamada al sistema; con el modo de sondeo, un hilo del kernel las recoge incluso sin esa llamada.

Procesamiento de datos

Hasta ahora hemos visto como llegan los datos, pero ¿Qué hace el servidor con los datos?.

El servidor trabaja en ticks, que son pasos discretos de simulación del videojuego. En cada tick el servidor procesa los comandos de los jugadores, ejecuta un paso en la física, revisa las reglas del juego y actualiza el estado de los objetos, después si algún cliente necesita una actualización, se toma una instantánea del mundo.

Tomando como ejemplo el motor Source de Valve, se usan por defecto 66 ticks por segundo, es decir, un tick equivale a 15 milisegundos. Ese es todo el tiempo que tenemos, todo el trabajo debe caber ahí. Usar más ticks da más precisión, pero obviamente exige más de la CPU y más ancho de banda. Aunque cabe aclarar que los ticks que se usan por segundo varían de juego a juego dependiendo de cuánta exactitud se requiera.

Una vez que se termine de procesar todo el trabajo, el servidor duerme hasta el siguiente tick, y aquí entra el sistema operativo. Dormir puede llevar a retrasos, ya que si nuestro tick es cada 15 ms como en el ejemplo anterior, el kernel no despierta al hilo en un instante preciso. Primero porque el reloj del sistema funciona a pasos, es decir, si el tiempo termina entre dos pasos se redondea hacia arriba. Segundo, aunque el tiempo ya se cumplió, el hilo pasa a la cola de listos y espera a que el planificador le dé una CPU libre. Con el ejemplo anterior, esto podría llevarnos a despertar en 15.3 ms.

Este retraso puede no parecer mucho, pero al acumularse tras miles de ticks el servidor se atrasaría varios cientos de milésimas de segundo, y la simulación correría más lenta que el reloj real, y provocaría lo que un jugador percibe como lag.

Una solución es no dormir durante un tiempo, sino hasta una hora fija. Con la función clock_nanosleep y la bandera TIMER_ABSTIME calculamos la hora exacta de cada tick

antes de iniciarlo, así si nos atrasamos en un tick, el siguiente tick duerme un poco menos, evitando que el error se acumule.

Redes TCP y UDP

En cuanto a la red hay dos opciones, TCP (Protocolo de Control de Transmisión), entrega todo en orden y sin pérdidas, haciendo que cuando un paquete se pierde, retiene los siguientes en una cola hasta que el primero se retransmita, es decir usa el bloqueo de cabeza de línea.

Digamos que el servidor envía el estado del mundo diez veces por segundo, un paquete en el instante 10.0, otro en el 10.1, otro en el 10.2. Si se pierde el paquete del instante 10.0 los paquetes 10.1 y 10.2 sí llegan al jugador, pero TCP no se los entrega al juego porque falta el primero. Con un ping alto, pedir el reenvío y recibirlo toma décimas de segundo. Para entonces el juego ya querrá mostrar el instante 10.3, y el paquete 10.0, es información vieja que no sirve.

En un juego competitivo nos conviene lo contrario, si se pierde un estado, se ignora y se usa el más reciente, esto es UDP (Protocolo de Datagramas de Usuario). A cambio UDP no da conexión ni fiabilidad.

Por lo que UDP es el más útil a la hora de la interacción entre jugadores, pero no sirve para sistemas que sí necesitan de cada paquete sin excepción. Un juego multijugador en realidad suele utilizar ambos sistemas, TCP se utiliza más para cuestiones como los mensajes por chat, matchmaking, carga de assets, etc.

Organización de procesos del mundo

Un servidor no calcula toda la física ni todas las partes de la simulación, el equipo del jugador se encarga de simular cosas como la renderización de los gráficos, reproducción de la música y efectos de sonido, la física no importante para la experiencia de juego o una física provisional. El servidor se encarga de validar las acciones, ejecuta la física, calcula la lógica del juego, y transmite el estado final del mundo a todo los jugadores.

Dicho esto, el servidor aún tiene de qué preocuparse incluso solo con su parte. Cuando varios hilos modifican simultáneamente la misma lista de jugadores, posiciones o elementos de la física, pueden aparecer condiciones de carrera, deadlock y problemas de contención, esto puede provocar inconsistencias en los datos y dar errores durante la simulación. Para evitar esto existen tres estrategias principales.

La primera estrategia es utilizar un hilo dueño de cada partida**.** Este hilo se encarga de ejecutar toda la simulación de su sala, incluyendo el movimiento de los jugadores, las colisiones, las acciones y las actualizaciones del mundo, sin necesidad de utilizar candados para proteger esos datos dentro de la simulación. El paralelismo se consigue ejecutando varias partidas al mismo tiempo, distribuyéndolas entre diferentes núcleos del procesador,

con `sched_setaffinity` se puede fijar cada hilo a un núcleo para no perder la caché al migrar .

La segunda estrategia es el paso de mensajes, haciendo que los hilos encargados de la red reciban los paquetes enviados por los jugadores y los colocan en una cola. Posteriormente, el hilo encargado de la simulación consume esos mensajes durante cada tick. Esto corresponde al patrón productor-consumidor.

La tercera estrategia es utilizar paralelismo dentro de un mismo tick**.** Esta opción es especialmente útil en videojuegos con mundos grandes o simulaciones complejas, donde una sola partida puede necesitar aprovechar varios núcleos del procesador. Para conseguirlo, se dividen las tareas en diferentes partes, por ejemplo, calculando la física de distintas regiones del mapa en paralelo. Sin embargo, es necesario utilizar mecanismos de sincronización, como mutex para proteger datos compartidos, variables de condición o colas bloqueantes para el paso de mensajes, y barreras (`pthread_barrier`) para asegurar que todos los hilos terminen una fase antes de comenzar la siguiente. De esta manera, se evita que un hilo lea información que otro todavía está modificando. Aunque se aprovechan más recursos del procesador, también aumenta la complejidad y el costo de la sincronización.

En general evitar candados reduce errores de concurrencia y contención en memoria, pero limita la potencia asignable a una sola instancia; paralelizar el interior del *tick* exprime el hardware al máximo, pero a costa de pagar un peaje en sincronización y complejidad.

Conclusión

Un servidor multijugador debe atender miles de conexiones y, al mismo tiempo, simular el mundo del juego a intervalos fijos de unos pocos milisegundos. Para lograrlo, el sistema operativo es muy importante. Un hilo por conexión puede resultar cómodo, pero su costo en memoria y en cambios de contexto lo hace poco eficiente a gran escala. A su vez, la precisión del temporizador y del planificador determina qué tan bien se cumple el tiempo de cada tick.

La concurrencia dentro del juego se simplifica al reducir el estado compartido. Para ello se asigna un hilo a cada partida o se comunican los hilos mediante colas, y la escala se consigue ejecutando muchas partidas en paralelo. En la red, UDP evita que un paquete perdido retrase los más recientes, aunque obliga a construir sobre él la gestión de conexiones y la fiabilidad que el juego necesite.

En conjunto, el sistema operativo no tiene por qué ser el cuello de botella de un servidor de videojuegos, siempre que se usen las herramientas correctas y se conozca la escala de jugadores.